TypeScript Fundamentals - Part 1

Introduction & Setup

What is TypeScript?

TypeScript is a programming language developed by Microsoft that builds on JavaScript by adding static type definitions. Think of TypeScript as JavaScript with superpowers. It allows you to catch errors during development rather than at runtime, making your code more reliable and easier to maintain.

TypeScript is a **superset** of JavaScript, which means any valid JavaScript code is also valid TypeScript code. However, TypeScript adds additional features that JavaScript does not have.

Key Features of TypeScript

TypeScript introduces several powerful features that enhance JavaScript development:

Static Type Checking: TypeScript checks your code for type errors before you run it. This catches mistakes early in the development process.

Type Annotations: You can explicitly specify what type of data a variable should hold, making your code self-documenting and easier to understand.

Enhanced IDE Support: Code editors can provide better autocomplete, navigation, and refactoring tools because they understand the types in your code.

Modern JavaScript Features: TypeScript supports the latest JavaScript features and can compile them down to older JavaScript versions for browser compatibility.

Object-Oriented Programming: TypeScript provides robust support for classes, interfaces, and other OOP concepts with type safety.

TypeScript vs JavaScript: Understanding the Difference

Let's compare the same code written in JavaScript and TypeScript to understand the fundamental difference:

JavaScript Example:

```
function addNumbers(a, b) {
    return a + b;
}

let result = addNumbers(5, 10);
console.log(result);

// This will not show an error in JavaScript
let wrongResult = addNumbers("5", "10");
console.log(wrongResult);
```

Output:

```
15
510
```

Notice how JavaScript concatenates the strings instead of adding numbers. No error is shown.

TypeScript Example:

```
function addNumbers(a: number, b: number): number {
    return a + b;
}

let result = addNumbers(5, 10);
console.log(result);

// TypeScript will show an error before running
// Error: Argument of type 'string' is not assignable to parameter of type 'number'
let wrongResult = addNumbers("5", "10");  // ❌ This causes a compile error
console.log(wrongResult);
```

Output:

```
15
```

> TypeScript prevents the error by catching it during compilation. The second function call will not compile.

In the TypeScript version, we specify that both parameters (`a` and `b`) must be numbers, and the function returns a number. When we try to pass strings, TypeScript immediately alerts us to the error, preventing bugs before the code runs.

Why Use TypeScript in Enterprise Applications?

TypeScript has become the standard choice for large-scale applications due to several compelling reasons:

Early Error Detection: Catching bugs during development is far cheaper than finding them in production. TypeScript's compiler identifies issues before deployment, reducing costly runtime errors.

Better Code Maintainability: Type annotations serve as inline documentation. When you return to code months later, or when a new team member joins, types make it clear what each function expects and returns.

Improved Refactoring: When you need to change how your code works, TypeScript ensures that all related code is updated correctly. The compiler will flag any inconsistencies across your codebase.

Team Collaboration: In large teams, TypeScript acts as a contract between different parts of the application. Developers know exactly what data structures to expect, reducing integration issues.

Scalability: As applications grow, JavaScript's flexibility can become a liability. TypeScript's structure helps manage complexity in codebases with thousands of files.

The TypeScript Compilation Process

TypeScript cannot run directly in browsers or Node.js. It must be **compiled** (also called **transpiled**) into JavaScript. Understanding this process is crucial for working with TypeScript.

Here's how the compilation process works:

1. **Write TypeScript Code:** You create `.ts` files with TypeScript syntax and type annotations.
2. **Run the TypeScript Compiler:** The TypeScript compiler (`tsc`) reads your `.ts` files.
3. **Type Checking:** The compiler checks all types and reports any errors it finds.
4. **Generate JavaScript:** If there are no blocking errors, the compiler produces `.js` files.

5. **Execute JavaScript:** The resulting JavaScript files can run in any JavaScript environment (browsers, Node.js, etc.).

> **Important Note:** Even if TypeScript finds type errors, it will usually still generate JavaScript files. This allows you to run your code during development, but you should fix all type errors before deploying to production.

Installing TypeScript

To start working with TypeScript, you need to install it on your system. TypeScript is distributed as an npm package, so you need Node.js and npm installed first.

Check if Node.js is installed:

```
node --version
npm --version
```

> **Output:**
>
> `v18.17.0`
>
> `9.6.7`
>
> Your version numbers may differ. Any recent version of Node.js will work.

Install TypeScript globally:

```
npm install -g typescript
```

> **Output:**
>
> `added 1 package in 2s`
>
> The -g flag installs TypeScript globally, making the `tsc` command available everywhere on your system.

Verify the installation:

```
tsc --version
```

> **Output:**
>
> `Version 5.3.3`
>
> This confirms TypeScript is installed and shows the installed version.

Compiling TypeScript Files

Once TypeScript is installed, you can compile your TypeScript files to JavaScript. Let's walk through a complete example.

Step 1: Create a TypeScript file

Create a file named `hello.ts` with the following content:

```
let message: string = "Hello, TypeScript!";
console.log(message);

function greet(name: string): string {
    return `Welcome, ${name}!`;
}

console.log(greet("Developer"));
```

Step 2: Compile the TypeScript file

```
tsc hello.ts
```

> **Output:**

> The compiler runs and creates a `hello.js` file in the same directory. If there are any errors, they will be displayed here.

Step 3: The generated JavaScript file (`hello.js`)

```
var message = "Hello, TypeScript!";
console.log(message);

function greet(name) {
    return "Welcome, ".concat(name, "!");
}

console.log(greet("Developer"));
```

> **Output:**
>
> Notice that all type annotations have been removed. The generated JavaScript is clean and compatible with any JavaScript environment.

Step 4: Run the JavaScript file

```
node hello.js
```

> **Output:**
>
> `Hello, TypeScript!`
> `Welcome, Developer!`

Understanding tsconfig.json

For real projects, manually compiling each file becomes impractical. TypeScript uses a configuration file called `tsconfig.json` to manage compiler options and determine which files to compile.

Creating a tsconfig.json file:

```
tsc --init
```

Output:

```
Created a new tsconfig.json with:
target: es2016
module: commonjs
strict: true
esModuleInterop: true
skipLibCheck: true
forceConsistentCasingInFileNames: true
```

This creates a default configuration file with sensible defaults.

Basic tsconfig.json structure:

```
{
  "compilerOptions": {
    "target": "ES2016",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "forceConsistentCasingInFileNames": true
  },
  "include": ["src/**/*"],
  "exclude": ["node_modules"]
}
```

Let's understand the key configuration options:

target: Specifies which version of JavaScript to compile to. `ES2016` means the output will use JavaScript features from 2016.

module: Defines the module system to use. `commonjs` is used for Node.js applications.

outDir: Specifies the folder where compiled JavaScript files will be placed. This keeps your source and compiled files separate.

rootDir: Defines where your TypeScript source files are located.

strict: Enables all strict type checking options. This is recommended for catching more potential errors.

include: Specifies which files to compile. `src/**/*` means all files in the src folder and its subfolders.

exclude: Specifies which files to ignore. Typically includes `node_modules` and build directories.

Using tsconfig.json:

Once you have a `tsconfig.json` file, you can simply run:

```
tsc
```

> **Output:**
>
> The compiler will use the settings in `tsconfig.json` to compile all files specified in the configuration.

Watch mode for development:

During development, you can use watch mode to automatically recompile when files change:

```
tsc --watch
```

> **Output:**
>
> ```
> Starting compilation in watch mode...
> Found 0 errors. Watching for file changes.
> ```
>
> The compiler stays running and recompiles files whenever you save changes.

Practical Example: Complete TypeScript Setup

Let's create a small project to demonstrate the complete setup process:

Step 1: Create project structure

```
mkdir my-typescript-project
cd my-typescript-project
npm init -y
npm install -g typescript
```

Step 2: Initialize TypeScript

```
tsc --init
```

Step 3: Create source folder and file

```
mkdir src
cd src
```

Create `src/calculator.ts`:

```
class Calculator {
    add(a: number, b: number): number {
        return a + b;
    }

    subtract(a: number, b: number): number {
        return a - b;
    }

    multiply(a: number, b: number): number {
        return a * b;
    }

    divide(a: number, b: number): number {
        if (b === 0) {
            throw new Error("Cannot divide by zero");
        }
```

```
        return a / b;
    }
}

const calc = new Calculator();
console.log("Addition: 10 + 5 =", calc.add(10, 5));
console.log("Subtraction: 10 - 5 =", calc.subtract(10, 5));
console.log("Multiplication: 10 * 5 =", calc.multiply(10, 5));
console.log("Division: 10 / 5 =", calc.divide(10, 5));
```

Step 4: Update tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2016",
    "module": "commonjs",
    "outDir": "./dist",
    "rootDir": "./src",
    "strict": true
  }
}
```

Step 5: Compile and run

```
tsc
node dist/calculator.js
```

Output:

```
Addition: 10 + 5 = 15
Subtraction: 10 - 5 = 5
Multiplication: 10 * 5 = 50
Division: 10 / 5 = 2
```

This example demonstrates a complete TypeScript workflow: writing typed code, compiling it, and executing the result. The Calculator class uses type annotations to ensure all operations receive and return numbers, preventing common errors.

Summary

In this section, we covered the foundations of TypeScript. You learned what TypeScript is and why it's valuable for building reliable applications. We explored the compilation process that transforms TypeScript into JavaScript, and you practiced installing TypeScript and setting up a project with proper configuration.

The key takeaways are that TypeScript adds type safety to JavaScript, catching errors during development rather than at runtime. The `tsc` compiler transforms your TypeScript code into JavaScript that can run anywhere, and the `tsconfig.json` file manages your project's compilation settings.

With this foundation in place, you're ready to explore TypeScript's type system in detail, which we'll cover in the next section.